

Memory Match

CS5450 Mobile Programming — Exercise 2

Flutter/Dart Implementation by Om

1. Overview

Memory Match is a card-flipping memory game built entirely in Flutter/Dart. The player selects a difficulty, then flips cards two at a time to find matching emoji pairs. The game tracks moves, matched pairs, and elapsed time. A win dialog appears when all pairs are found.

2. Project Structure

Path	Description
lib/main.dart	App entry point + HomeScreen with difficulty selection
lib/game_screen.dart	Main game UI: stats bar, card grid, restart button
lib/game_controller.dart	Game logic: shuffling, flip tracking, match detection, timer
lib/flip_card_widget.dart	3D flip animation widget using AnimationController
lib/card_model.dart	Immutable CardModel data class
pubspec.yaml	Flutter dependencies and project metadata
android/	Android-specific build files (Gradle, Manifest, Kotlin)
test/widget_test.dart	Basic widget smoke test
README.pdf	This document

3. Prerequisites

Flutter SDK: 3.10 or higher — <https://flutter.dev/docs/get-started/install>

Android Studio: Hedgehog (2023.1.1) or newer

Android SDK: API 21+ (minSdk), API 34 recommended (targetSdk)

Dart SDK: Included with Flutter ($\geq 3.0.0$)

JDK: Java 11 or 17 (bundled with Android Studio)

4. How to Configure and Run in Android Studio

Step 1 — Open the Project

Launch Android Studio. Choose **File** → **Open** and navigate to the **memory_match** folder (the root that contains *pubspec.yaml*). Android Studio will detect it as a Flutter project automatically.

Step 2 — Install Flutter & Dart Plugins (first time only)

Go to **File** → **Settings** → **Plugins** and search for "Flutter". Install it; this also installs the Dart plugin. Restart Android Studio when prompted.

Step 3 — Set the Flutter SDK Path

Go to **File** → **Settings** → **Languages & Frameworks** → **Flutter** and set the Flutter SDK path (e.g. C:\flutter or /home/user/flutter). Click Apply.

Step 4 — Get Dependencies

Open the Terminal tab inside Android Studio and run:

```
flutter pub get
```

Step 5 — Create/Start an Emulator

Go to **Tools** → **Device Manager** → **Create Device**. Choose Pixel 6 (or any), select a system image (API 33 or 34), and click Finish. Press the Play button to start it.

Step 6 — Run the App

Select your emulator from the device dropdown at the top of Android Studio, then click the green Run button (Shift+F10). The app will build and launch on the emulator.

Or via terminal:

```
flutter run
```

5. Features

- Three difficulty levels: Easy (4x3 = 6 pairs), Medium (4x4 = 8 pairs), Hard (4x5 = 10 pairs)
- Smooth 3D card flip animation using AnimationController + Matrix4 perspective transform
- Real-time stats: move counter, matched pairs tracker, elapsed timer
- Color-coded card backs (6 rotating colors) for visual appeal
- Win dialog with time and move summary
- Restart button to replay at same difficulty; Home button to change difficulty
- Responsive layout — works on phones and tablets in portrait mode
- No external packages required beyond Flutter SDK

6. Architecture

The app uses a simple **ChangeNotifier (MVVM-lite)** pattern:

Layer	File	Responsibility
Model	card_model.dart	Holds card state (emoji, isFlipped, isMatched)
Controller	game_controller.dart	Game logic, timer, match detection, notifyListeners()
View	game_screen.dart	Listens to controller, rebuilds grid on state change
Widget	flip_card_widget.dart	Stateful widget with flip animation; reads CardModel

7. App Screens

Home Screen: Purple gradient background with game title and three difficulty buttons (Easy/Medium/Hard).

Game Screen: AppBar with title, stats bar showing Moves/Pairs/Time, 4-column emoji grid, Restart button.

Card Back: Colorful card showing '?' — 6 rotating background colors per card position.

Card Front: White card showing emoji; matched cards get green border and light green background.

Win Dialog: Celebration emoji, 'You Win!' heading, time/moves/pairs summary, Play Again + Menu buttons.

8. Troubleshooting

Problem: flutter pub get fails

Solution: Ensure you have internet access and that pubspec.yaml is in the project root.

Problem: Gradle sync error

Solution: Check that Android SDK is installed via SDK Manager. Run flutter doctor for diagnostics.

Problem: Emulator not showing

Solution: Start it from Device Manager first, then re-select from the device dropdown.

Problem: flutter: command not found

Solution: Add Flutter's bin folder to your system PATH and restart Android Studio.

Problem: App crashes on launch

Solution: Run flutter clean then flutter pub get and try again.